ACO-Based Edge Detection

Ant Colony Optimization for Image Edge Detection

Evaluated on the Berkeley Segmentation Dataset (BSDS300)

Mini Project - Image Analysis & Computer Vision

Team Members: [Update Names Here]

Guide: [Professor Name]

Institution: [College / University Name]

Problem Statement

Edge detection is a fundamental operation in image analysis - identifying boundaries where pixel intensity changes sharply.

Challenges with Traditional Methods:

- Fixed-kernel methods (Sobel, Canny) cannot adaptively respond to varying edge strengths
- Pixel-wise operators produce fragmented, disconnected edge contours
- Sensitive to noise without careful parameter tuning

Our Approach:

- Apply Ant Colony Optimization (ACO) - a swarm intelligence metaheuristic
- Artificial ants traverse the pixel grid, depositing pheromone on edge locations
- Pheromone accumulates along true edge contours over multiple iterations
- Evaluate against human-annotated ground truth from BSDS300 (300 images)

Literature Survey

[1] Tian, Yu & Xie (2008) - IEEE CEC

- Pioneered ACO for image edge detection. Ants traverse pixel grids using pheromone and gradient heuristics. Competitive with Sobel and Canny.

[2] Nezamabadi-pour et al. - MRF-Based Image Segmentation

- Extended ACO to MRF-based segmentation, showing pheromone-guided traversal captures spatial context beyond local gradient information.

[3] IJETT V71I9P233 - Edge Detection Using ACO

- Comprehensive survey comparing ACO with classical methods. Analyzes parameter sensitivity. ACO produces smoother, more connected edges.

Biological Inspiration

Real Ant Behavior

- Ants deposit pheromone on paths while foraging
- Shorter, better paths accumulate more pheromone
- Other ants prefer pheromone-rich paths
- Old pheromone evaporates over time
- Collective behavior -> emergent optimization

Our Analogy

- Image pixels = graph nodes
- 8-connected neighbors = possible moves
- Gradient magnitude = heuristic desirability
- Pheromone matrix = collective edge memory
- Edge contours emerge from pheromone accumulation

Mathematical Formulation

Transition Probability

Each ant at pixel (i,j) chooses its next position from the 8-connected neighborhood:

```
P(i,j)  =  [ t(i,j)^a  *  n(i,j)^b ]  /  Sum [ t(m,n)^a  *  n(m,n)^b ]
```

Pheromone Update

```
t(i,j)  <-  (1 - rho) * t(i,j)  +  Delta_t(i,j)
```

Where:

- t(i,j) = pheromone level n(i,j) = heuristic (Sobel gradient)
- a = pheromone weight b = heuristic weight rho = evaporation rate

System Architecture & Pipeline

Step 1	>	Step 2	>	Step 3	>	Step 4	>	Step 5	>	Step 6
Input Image (Grayscale)		Sobel Gradient n(i,j)		Init Pheromone t = 0.1		Ant Traversal K ants, S steps		Pheromone Update		Edge Map [0, 255]

Software Stack

- Python >= 3.13 NumPy >= 2.4 OpenCV >= 4.13 tqdm

Code Structure

- main.py - AntColonyEdgeDetector class (core ACO algorithm)
- process_images.py - Batch processing pipeline with CLI support

Core Algorithm - Code

```
class AntColonyEdgeDetector:
    def run_detection(self):
        self._initialize_matrices()           # Sobel gradient + pheromone init

        for iteration in range(self.num_iterations):
            # Place K ants at random pixel positions
            ant_rows = np.random.randint(0, H, self.num_ants)
            ant_cols = np.random.randint(0, W, self.num_ants)

            for step in range(steps_per_ant):
                for ant in range(self.num_ants):
                    # Transition probability: P = t^a * n^b
                    # Choose next pixel from 8-connected neighbors
                    # Deposit pheromone proportional to gradient

            # Global pheromone update with evaporation
            t = (1 - rho) * t + delta_t

        # Normalize pheromone matrix -> edge map [0, 255]
        # Dilate edges with 3x3 kernel for visibility
        return dilated_edge_map
```

Dataset - BSDS300

Berkeley Segmentation Dataset - 300 natural images with human-annotated ground truth

300	200	100	5-7
Natural Images	Training Set	Test Set	Annotators/Image

Dataset Features

- Diverse scenes: animals, landscapes, people, objects, urban
- Multiple independent human segmentations per image
- Standard benchmark in edge detection and segmentation literature

Our Processing

- All images converted to grayscale
- .seg files parsed to binary edge maps via Sobel
- 3-panel dashboard visualization per image

Hyperparameters

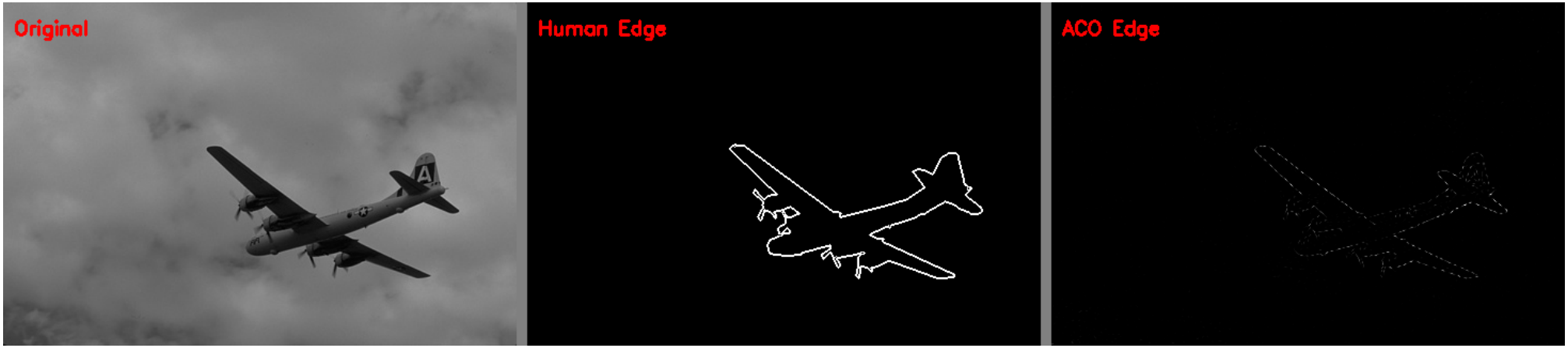
Parameter	Symbol	Default	Batch	Description
`num_ants`	K	2,048	8,192	Ants per iteration
`num_iterations`	T	20	50	ACO iterations
`alpha`	a	1.0	1.0	Pheromone influence weight
`beta`	b	2.0	2.0	Heuristic (gradient) weight
`decay_rate`	rho	0.1	0.1	Pheromone evaporation rate
`steps_per_ant`	S	10	10	Steps each ant takes

Key insight: b > a favors gradient-guided exploration, producing edges aligned with true intensity transitions. Batch uses 8192 ants and 50 iterations for production quality.

Results - Sample Outputs (Set 1)

Each dashboard: Original | Human Ground Truth | ACO Edge Map

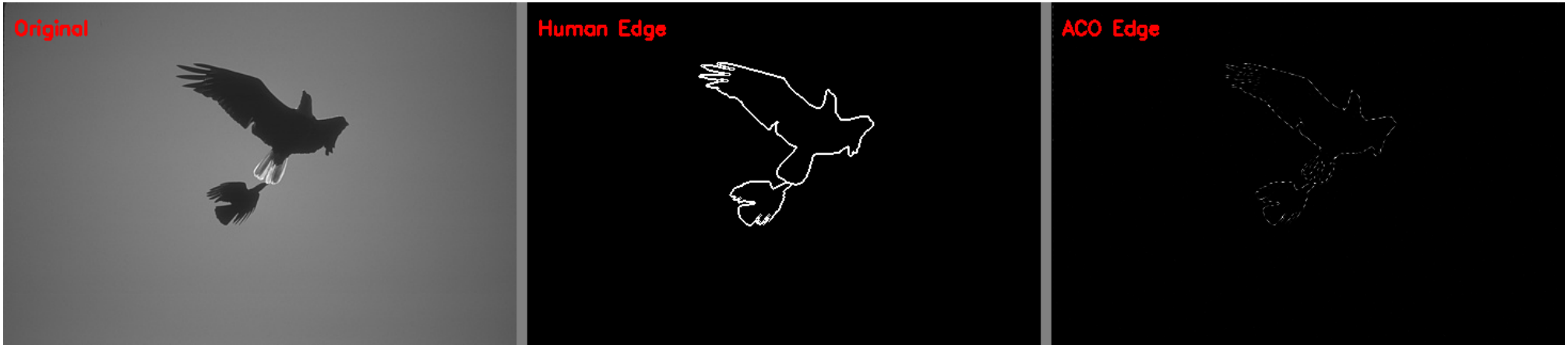
BSDS300 #100075 - Bears at water's edge



BSDS300 #3096 - Military aircraft in flight

Results - Sample Outputs (Set 2)

BSDS300 #42049 - Bird on branch



BSDS300 #135069 - Eagles in flight

Strengths & Limitations

Strengths

- Adaptive: pheromone reinforcement concentrates on true edges
- Connected edges: ant traversal produces continuous contours
- Noise tolerance: collective behavior averages out noise
- Flexible: tunable via a, b, rho, ant count parameters
- No training required: pure optimization approach

Limitations

- Computational cost: per-ant loops are expensive
- Parameter sensitivity: poor a/b produce weak edges
- Thinner edges: ACO edges sparser than ground truth
- No multi-scale: single-resolution Sobel heuristic
- Sequential bottleneck: hard to parallelize on CPU

ACO vs. Traditional Methods

Property	Sobel	Canny	ACO (Ours)
Approach	Fixed 3x3 kernels	Multi-stage pipeline	Swarm intelligence
Adaptivity	None	Threshold-based	Pheromone-guided
Edge Continuity	Low (pixel-wise)	Medium (hysteresis)	High (ant paths)
Noise Handling	Poor	Good (Gaussian blur)	Good (collective avg.)
Speed	Very fast	Fast	Slow (iterative)
Parameters	Kernel size	2 thresholds + sigma	a, b, rho, K, T, S

Key takeaway: ACO trades computational speed for adaptivity and edge continuity. Best suited for applications where edge quality matters more than real-time performance.

Conclusion & Future Work

Conclusions

- Successfully implemented ACO edge detection
- Evaluated on all 300 BSDS300 images
- Produces coherent, continuous edge contours
- Fully unsupervised - no training or labels required
- Dashboard visualizations enable direct comparison with human ground truth

Future Work

- GPU acceleration via CUDA/OpenCL
- Multi-scale heuristics using Gaussian pyramids
- Hybrid ACO + Canny for precise edges
- Quantitative metrics: F1, ODS, OIS scores
- Auto-tune parameters via PSO / genetic algorithms

References

[1] Tian, J., Yu, W., & Xie, S. (2008). "An Ant Colony Optimization Algorithm for Image Edge Detection." IEEE CEC, pp. 751-756.

[2] Nezamabadi-pour, H., et al. "Ant Colonies for MRF-Based Image Segmentation." J. Applied Sciences.

[3] "Edge Detection Using Ant Colony Optimization." IJETT, V71I9P233.

[4] Martin, D., Fowlkes, C., Tal, D., & Malik, J. (2001). "A Database of Human Segmented Natural Images." ICCV.

[5] Dorigo, M. & Stutzle, T. (2004). Ant Colony Optimization. MIT Press.

[6] Canny, J. (1986). "A Computational Approach to Edge Detection." IEEE TPAMI, 8(6), pp. 679-698.

Thank You

Questions & Discussion